Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №2

«Таблицы»

Выполнил:
студент группы 3824Б1ФИ2
Ненев А.Е.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 7
3.1. Описание структуры программы 7
3.2. Описание структур данных 9
3.3. Описание алгоритмов 14
4. Результаты экспериментов и тестирования 17
Заключение 19
Литература 21

Введение

В современном программировании эффективное хранение и обработка данных являются ключевыми задачами. Одной из фундаментальных областей информатики является работа со структурами данных, которые определяют, как данные организуются, хранятся и обрабатываются.

Предметная область данного проекта - структуры данных для хранения и манипуляции объектами произвольных типов, в частности, многочленами (полиномами) от нескольких переменных.

1. Постановка задачи

Цель работы — Разработать программу, которая предоставляет возможность хранить данные в таблицах трёх типов с подсчётом количества произведённых операций с момента создания:

1) Неупорядоченная таблица (на массиве или векторе)

2) Упорядоченная таблица на сбалансированном бинарном дереве поиска (АВЛ или красно-чёрные)

3) Хэш-таблица

Таблицы должны поддерживать хранение произвольных данных, но в качестве данных в пользовательском консольном приложении, при тестировании и проведении экспериментов необходимо использовать полиномы.

Работоспособность программы необходимо проверить с помощью Google Test-ов. Кроме того, необходимо разработать пользовательское консольное приложение.

Особенности реализации:

1. Работа происходит сразу с таблицами всех типов.

2. Ключ – строка, сравнение лексикографическое.

3. Хэш-таблица должна быть реализована с одним из способов борьбы с коллизиями.

4. Пользователь может положить в таблицу свои данные, с заданием ему некоторого наименования (ключа).

5. Пользователь может запросить (выдать или извлечь) из таблицы данные с заданным наименованием (ключом).

6. В пользовательском консольном приложении должна быть возможность производить операции с полиномами, а также размещать полученный результат в таблицу.

7. Во время работы пользовательского консольного приложения происходит логирование количества произведенных операций. Пример: Пользователь ввёл свой полином и запросил положить его в таблицу, программа считает и записывает в файл, какое действие произошло и сколько операций потребовалось на нее у разных типов таблиц. Аналогично с поиском и извлечением. После завершения работы в файл добавляется общее число операций для каждого действия и в целом у разных типов таблиц.

2. Руководство пользователя

После запуска приложения отображается главное меню со следующими командами:

1. Insertion
2. Deletion
3. Search
4. Print
5. Polynomial operations
6. Operation statistics
7. Statistics reset
8. Exit

Описание команд:

Insertion (Вставка):

Введите ключ (строка) и значение.

Для Polinomial значение вводится в формате: коэффициент степень коэффициент степень...

Данные сохраняются во все три таблицы одновременно.

Deletion (Удаление):

Введите ключ для удаления.

Запись удаляется из всех трех таблиц.

Search (Поиск):

Введите ключ для поиска.

Программа выводит значение из каждой таблицы (если найдено) или сообщение об отсутствии.

Print (Печать):

Выводит содержимое всех трех таблиц в консоль.

Polynomial operations (Операции с полиномами):

Доступно только при выборе типа данных Polinomial.

Меню выбора операции: сложение, вычитание, умножение, умножение на скаляр.

Ввод полинома: Предлагается выбор:

Ввести новый полином с консоли.

Выбрать существующий полином из таблицы по ключу.

После выполнения операции результат выводится на экран и предлагается ввести ключ для сохранения результата в таблицы.

Operation statistics (Статистика операций):

Выводит количество выполненных операций для каждой таблицы (вставка, удаление, поиск, печать) и общее количество внутренних операций.

Statistics reset (Сброс статистики):
Обнуляет счетчики операций.

Exit (Выход):
Завершает работу программы и записывает итоговую статистику в файл log.txt.

При этом все действия пользователя записываются в файл log.txt с указанием времени, типа операции и количества выполненных внутренних операций.

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из следующих модулей (файлов):

polinomial.h / polinomial.cpp — модуль класса Polinomial.

Содержит структуру Monomial (одночлен) и класс MyList (список одночленов).

Класс Polinomial реализует арифметические операции и ввод/вывод.

hash_table.h — шаблонный класс хэш-таблицы, реализованный на методе цепочек.

unordered_table.h — шаблонный класс неупорядоченной таблицы.

Реализован на хранении данных в std::vector.

avl_tree.h — шаблонный класс АВЛ-дерева. Реализует сбалансированное бинарное дерево поиска.

table_manager.h — класс управления интерфейсом.
Связывает пользовательский ввод с операциями над таблицами.

Управляет логированием и статистикой.

app.cpp — точка входа в программу.

tests.cpp — модульные тесты (Google Test).

Состав классов и функций:

Polinomial Module:

struct Monomial: хранит коэффициент и степень.

class MyList: реализация списка для хранения одночленов (итераторы, вставка, сортировка).

class Polinomial: обертка над списком, реализует +, -, *, ==, <<, >>.

HashTable Module:

struct HashTableNode: узел списка в цепочке.

class HashTable: шаблонный класс с методами Find, EmplaceBack, Erase.

UnorderedTable Module:

struct TableNode: узел данных.

class UnorderedTable: шаблонный класс на векторе.

AvlTree Module:

struct AvlNode: узел дерева с высотой.

class AvlTree: шаблонный класс с балансировкой, методами вставки, удаления, поиска.

TableManager Module:

class TableManager: управляет тремя таблицами, обрабатывает ввод, логирует действия.

3.2. Описание структур данных

3.2.1. Шаблонный класс HashTableNode (Узел хэш-таблицы)

Назначение: Узел хэш-таблицы (метод цепочек).

Структура:

- std::string key - Ключ
- ValType value - Значение
- HashTableNode<ValType>* next - Указатель на следующий узел в цепочке

Методы:

1. HashTableNode() — конструктор по умолчанию.
   - Инициализирует next = nullptr.
2. HashTableNode(const std::string& key, const ValType& val) — параметризованный конструктор.
   - Инициализирует ключ, значение и next = nullptr.

3.2.2. Шаблонный класс HashTable (Хэш-таблица)

Назначение: Реализация хэш-таблицы с методом цепочек.

Поля:

- std::vector<HashTableNode<ValType>*> data_ — вектор указателей на цепочки.
- std::size_t size_ — размер таблицы (количество корзин).
- int hash_counters_ — счетчик внутренних операций.

Методы:

1. **HashTable(std::size_t size)** — конструктор.
   - Создает таблицу заданного размера, инициализирует вектор nullptr.
2. **HashTable(const HashTable& other)** — конструктор копирования.
   - Создает глубокую копию таблицы.
3. **operator=(const HashTable& other)** — оператор присваивания.
   - Присваивает таблицу с освобождением старой памяти.
4. **~HashTable()** — деструктор.
   - Удаляет все узлы и освобождает память.
5. **Find(const std::string& key)** — поиск по ключу.
   - Возвращает указатель на значение или nullptr, если ключ не найден.
6. **EmplaceBack(const std::string& key, const ValType& value)** — вставка.
   - Добавляет пару ключ-значение (или обновляет значение при существующем ключе).
7. **Erase(const std::string& key)** — удаление.
   - Удаляет запись по ключу.
8. **Print() const** — вывод таблицы.
   - Выводит все записи в консоль.
9. **PrintOperations() const** — вывод статистики.
   - Выводит количество выполненных операций.
10. **Reset()** — сброс статистики.
    - Обнуляет счетчик операций.
10. **Hash(const std::string& key)** — хэш-функция (приватная).
    - Вычисляет индекс корзины для ключа (алгоритм DJB2).

11. **CopyChain(...)** — копирование цепочки (приватная).
    - Рекурсивно копирует связный список узлов.
12. **GetOperationNumber() const** — получение счетчика.
    - Возвращает текущее значение счетчика операций.

3.2.3. Шаблонный класс TableNode (Узел неупорядоченной таблицы)

Назначение: Узел неупорядоченной таблицы (вектор).

Структура (шаблон):

std::string key - Ключ

ValType value - Значение

Методы:

1. TableNode(const std::string& key, const ValType& val) — конструктор.
    - Инициализирует ключ и значение.

3.2.4. Шаблонный класс UnorderedTable (Неупорядоченная таблица)

Назначение: Реализация таблицы на основе динамического массива (вектора).

Поля:

- std::vector<TableNode<ValType>> data_ — вектор узлов.
- int unordered_counters_ — счетчик внутренних операций.

Методы:

1. **UnorderedTable()** — конструктор по умолчанию.
    - Создает пустую таблицу.
2. **UnorderedTable(const UnorderedTable& other)** — конструктор копирования.
    - Создает глубокую копию таблицы.
3. **operator=(const UnorderedTable& other)** — оператор присваивания.
    - Присваивает таблицу.
4. **EmplaceBack(const std::string& key, const ValType& value)** — вставка.
    - Добавляет пару ключ-значение (или обновляет значение при существующем ключе).
5. **Erase(const std::string& key)** — удаление.
    - Удаляет запись по ключу.
6. **Find(const std::string& key)** — поиск по ключу.
    - Возвращает указатель на значение или nullptr.
7. **Print() const** — вывод таблицы.
    - Выводит все записи в консоль.
8. **PrintOperations() const** — вывод статистики.
    - Выводит количество выполненных операций.
9. **GetOperationNumber() const** — получение счетчика.
    - Возвращает текущее значение счетчика операций.
10. **Reset()** — сброс статистики.
    - Обнуляет счетчик операций.

3.2.5. Шаблонный класс AvlNode (Узел АВЛ-дерева)

Назначение: Узел АВЛ-дерева с хранением высоты поддерева.
Структура (шаблон):
std::string key - Ключ
ValType value - Значение
int height - Высота поддерева
AvlNode* left - Указатель на левое поддерево
AvlNode* right - Указатель на правое поддерево
Методы:

1. **AvlNode(const std::string& k, const ValType& val)** — конструктор.
    - Инициализирует узел, устанавливает высоту 1, указатели в nullptr.

3.2.6. Шаблонный класс AvlTree **(АВЛ-дерево)**
Назначение: Реализация сбалансированного бинарного дерева поиска.
Поля:

- int avl_counters_ — счетчик внутренних операций.
- AvlNode<ValType>* root_ — корень дерева.

Методы:

1. **AvlTree()** — конструктор по умолчанию.
    - Создает пустое дерево (root_ = nullptr).
2. **AvlTree(const AvlTree& other)** — конструктор копирования.
    - Создает глубокую копию дерева.
3. **operator=(const AvlTree& other)** — оператор присваивания.
    - Присваивает дерево.
4. **~AvlTree()** — деструктор.
    - Удаляет все узлы дерева.
5. **Find(const std::string& key)** — поиск по ключу.
    - Возвращает указатель на значение или nullptr.
6. **EmplaceBack(const std::string& str, const ValType& val)** — вставка.
    - Добавляет узел с автоматической балансировкой.
7. **Erase(const std::string& str)** — удаление.
    - Удаляет узел с автоматической балансировкой.
8. **Print() const** — вывод дерева.
    - In-order обход (выводит отсортированные записи).
9. **PrintOperations() const** — вывод статистики.
    - Выводит количество выполненных операций.
10. **GetOperationNumber() const** — получение счетчика.
    - Возвращает текущее значение счетчика операций.
11. **Reset()** — сброс статистики.
    - Обнуляет счетчик операций.
12. **GetHeight(AvlNode<ValType>* node)** — получение высоты узла (приватная).
    - Возвращает высоту поддерева или 0 для nullptr.
13. **HeightDiff(AvlNode<ValType>* node)** — вычисление баланс-фактора (приватная).
    - Возвращает разность высот правого и левого поддеревьев.

14. **RecountHeight(AvlNode<ValType>* node)** — пересчет высоты (приватная).
    - Обновляет высоту узла после изменений.
15. **RightRotation(AvlNode<ValType>* parent)** — правый поворот (приватная).
    - Выполняет правый поворот для балансировки.
16. **LeftRotation(AvlNode<ValType>* parent)** — левый поворот (приватная).
    - Выполняет левый поворот для балансировки.
17. **BigLeftRotation(AvlNode<ValType>* parent)** — двойной левый поворот (приватная).
    - Выполняет LR-поворот.
18. **BigRightRotation(AvlNode<ValType>* parent)** — двойной правый поворот (приватная).
    - Выполняет RL-поворот.
19. **Balance(AvlNode<ValType>* node)** — балансировка узла (приватная).
    - Проверяет баланс-фактор и выполняет повороты при необходимости.
20. **FindMin(AvlNode<ValType>* node)** — поиск минимального узла (приватная).
    - Находит узел с минимальным ключом в поддереве.
21. **DeleteMin(AvlNode<ValType>* node)** — удаление минимального узла (приватная).
    - Удаляет минимальный узел и балансирует дерево.
22. **PrivateErase(AvlNode<ValType>* node, const std::string& str)** — удаление узла (приватная).
    - Рекурсивно удаляет узел с балансировкой.
23. **PrivateEmplaceBack(AVL_Node<ValType>* node, std::string str, ValType val)** — вставка узла (приватная).
    - Рекурсивно вставляет узел с балансировкой.
24. **PrivatePrint(AvlNode<ValType>* node) const** — вывод дерева (приватная).
    - In-order обход для вывода.
25. **FindNode(const std::string& key, AvlNode<ValType>* node)** — поиск узла (приватная).
    - Рекурсивный поиск узла по ключу.
26. **Clear(AvlNode<ValType>* node)** — очистка дерева (приватная).
    - Рекурсивное удаление всех узлов.

3.2.7. Класс TableManager (Менеджер таблиц)

Назначение: Управление пользовательским интерфейсом, таблицами и логированием.

Поля:

- HashTable<ValType> hash_table_ — хэш-таблица.
- UnorderedTable<ValType> unordered_table_ — неупорядоченная таблица.
- AvlTree<ValType> avl_tree_ — АВЛ-дерево.
- std::ofstream log_file_ — лог-файл.
- MethodStats hash_stats_ — статистика хэш-таблицы.
- MethodStats unordered_stats_ — статистика неупорядоченной таблицы.

- MethodStats avl_stats_ — статистика АВЛ-дерева.

Вложенная структура:

- MethodStats — структура для хранения статистики по методам:
    - int emplace_count — количество вставок.
    - int erase_count — количество удалений.
    - int find_count — количество поисков.
    - int print_count — количество печатей.

Методы:

1. **TableManager()** — конструктор.
    - Инициализирует таблицы, открывает лог-файл.
2. **~TableManager()** — деструктор.
    - Записывает итоговую статистику, закрывает лог-файл.
3. **Run()** — запуск приложения.
    - Главный цикл обработки меню.
4. **Log(const std::string& message, int operations)** — логирование.
    - Записывает сообщение в лог-файл с указанием операций.
5. **HandleEmplace()** — обработка вставки.
    - Читает ключ и значение, вставляет во все таблицы.
6. **HandleErase()** — обработка удаления.
    - Читает ключ, удаляет из всех таблиц.
7. **HandleFind()** — обработка поиска.
    - Читает ключ, ищет во всех таблицах, выводит результат.
8. **HandlePrint()** — обработка печати.
    - Выводит содержимое всех таблиц.
9. **HandlePolynomialOperations()** — обработка операций с полиномами.
    - Меню выбора операции, ввод полиномов, выполнение операции, сохранение результата.
10. **ReadPolynomial(const std::string& prompt)** — чтение полинома.
    - Читает полином с консоли в формате "коэф степень ...".
11. **SelectPolynomial(const std::string& prompt)** — выбор полинома.
    - Предлагает выбрать полином из таблицы или ввести новый.
12. **ShowStats()** — вывод статистики.
    - Выводит статистику по методам и общее количество операций.
13. **ResetStats()** — сброс статистики.
    - Обнуляет все счетчики операций.
14. **LogFinalStats()** — запись итоговой статистики.
    - Записывает в лог количество вызовов каждого метода.

3.3. Описание алгоритмов

3.3.1. Алгоритмы хэш-таблицы (HashTable)

Структура данных: Вектор указателей на связные списки (std::vector<HashTableNode<ValType>*>), метод цепочек для разрешения коллизий.

- **Метод** Hash(key)
  - **Описание:** Полиномиальная хэш-функция, инициализация res = 5381, для каждого символа res = res * 33 + char.
  - **Сложность по времени:** O(n), где n — длина ключа.
  - **Сложность по памяти:** O(1).
- **Метод** EmplaceBack(key, value)
  - **Описание:** Вычисляет хэш-индекс, проверяет наличие ключа в цепочке, добавляет новый узел в начало цепочки.
  - **Сложность по времени:** O(1) в среднем (при равномерном распределении), O(n) в худшем (все коллизии в одной цепочке).
  - **Сложность по памяти:** O(1).
- **Метод** Find(key)
  - **Описание:** Вычисляет хэш-индекс, ищет ключ в цепочке линейным поиском.
  - **Сложность по времени:** O(1) в среднем, $O(n)$ в худшем.
  - **Сложность по памяти:** O(1).
- **Метод** Erase(key)
  - **Описание:** Вычисляет хэш-индекс, ищет ключ в цепочке, удаляет узел.
  - **Сложность по времени:** O(1) в среднем, $O(n)$ в худшем.
  - **Сложность по памяти:** O(1).

3.3.2. Алгоритмы неупорядоченной таблицы (UnorderedTable)

Структура данных : std::vector<TableNode<ValType>> (динамический массив).

- **Метод** EmplaceBack(key, value)
  - **Описание:** Проверяет наличие ключа в векторе (линейный поиск), добавляет элемент в конец вектора.
  - **Сложность по времени:** O(n) в худшем (проверка на дубликаты) O(1) в среднем.
  - **Сложность по памяти:** O(1)
- **Метод** Find(key)
  - **Описание:** Линейный поиск элемента по ключу.
  - **Сложность по времени:** O(n).
  - **Сложность по памяти:** O(1).
- **Метод** Erase(key)
  - **Описание:** Поиск элемента, замена удаляемого элемента последним, в неупорядоченной таблице порядок не важен – экономит время и не сдвигает элементы вектора.
  - **Сложность по времени:** O(n).
  - **Сложность по памяти:** O(1).

3.3.3. Алгоритмы АВЛ-дерева (AvlTree)

Структура данных: Сбалансированное бинарное дерево поиска, где каждый узел хранит высоту поддерева.

- **Метод** EmplaceBack(key, value)
  - **Описание:** Рекурсивный поиск места для вставки, вставка узла, балансировка дерева.
  - **Алгоритм балансировки:** Вычисление баланс-фактора (height(right) - height(left)), выполнение поворотов при |balance| > 1.
  - **Сложность по времени:** O(logn)
  - **Сложность по памяти:** O(1) (добавление узла).
- **Метод** Find(key)
  - **Описание:** Рекурсивный поиск узла в дереве.
  - **Сложность по времени:** O(logn)
  - **Сложность по памяти:** O(1)
- **Метод** Erase(key)
  - **Описание:** Рекурсивный поиск узла, удаление с учетом балансировки.
  - **Алгоритм:** Если у удаляемого узла два потомка, заменяется на минимальный узел правого поддерева.
  - **Сложность по времени:** O(logn)
  - **Сложность по памяти:** O(1)*O*(1).
- **Методы поворотов (**RightRotation**,** LeftRotation**)**
  - **Описание:** Изменение структуры дерева для восстановления баланса.
  - **Сложность по времени:** O(1).
  - **Сложность по памяти:** O(1).
- **Метод** Balance()
  - **Описание:** Проверяет баланс-фактор и выполняет необходимые повороты.
  - **Сложность по времени:** O(1).
  - **Сложность по памяти:** O(1).
- **Метод** Print()
  - **Описание:** In-order обход дерева (левый корень правый).
  - **Сложность по времени:** O(n).
  - **Сложность по памяти:** O(logn) (стек рекурсии).

3.3.4. Алгоритмы класса TableManager

Структура данных: Содержит три таблицы, лог-файл, счетчики статистики.

- **Метод** HandleEmplace()
  - **Описание:** Вставка данных во все три таблицы одновременно.
  - **Сложность по времени:** Зависит от таблиц (вставка в три таблицы).
  - **Сложность по памяти:** O(1).
- **Метод** HandleFind()
  - **Описание:** Поиск данных во всех трех таблицах.
  - **Сложность по времени:** Зависит от таблиц (поиск в трех таблицах).
  - **Сложность по памяти:** O(1).
- **Метод** HandlePolynomialOperations()
  - **Описание:** Выполнение арифметических операций над полиномами.

    - **Сложность по времени:** Зависит от операции (сложение O(n*logn) умножение O(n^2)).
    - **Сложность по памяти:** Зависит от размера полиномов.
- **Метод** Log()
    - **Описание:** Запись операции в лог-файл.
    - **Сложность по времени:** O(1).
    - **Сложность по памяти:** O(1).

4. Результаты экспериментов и тестирования

Для проверки корректности работы программы были написаны модульные тесты с использованием фреймворка Google Test. Тесты разделены на две категории: **тесты** и **эксперименты производительности**.

Тесты

1. Тесты для HashTable<Polinomial>:

- EmplaceAndFind: проверка вставки и поиска полиномов
- EmplaceReplace: проверка обновления значения при существующем ключе
- Erase: проверка удаления полиномов
- MultipleOperations: проверка последовательности операций
- Statistics: проверка работы счетчиков операций
- EmptyTable: проверка работы с пустой таблицей
- CopyConstructor: проверка конструктора копирования
- AssignmentOperator: проверка оператора присваивания

2. Тесты для UnorderedTable<Polinomial>:

- Аналогичный набор тестов, как для HashTable

3. Тесты для AvlTree<Polinomial>:

- Аналогичный набор тестов, как для HashTable
- Дополнительно: BalanceCheck — проверка балансировки дерева при вставке 7 элементов

Эксперименты производительности

Для каждой структуры данных проведено 3 эксперимента на 10 000 случайных полиномов:

1. HashTable<Polinomial>:

- Insert10000Polynomials: вставка 10 000 полиномов
- Search10000Polynomials: поиск 10 000 полиномов
- Erase10000Polynomials: удаление 10 000 полиномов

2. UnorderedTable<Polinomial>:

- Insert10000Polynomials: вставка 10 000 полиномов
- Search10000Polynomials: поиск 10 000 полиномов
- Erase10000Polynomials: удаление 10 000 полиномов

3. AvlTree<Polinomial>:

- Insert10000Polynomials: вставка 10 000 полиномов
- Search10000Polynomials: поиск 10 000 полиномов
- Erase10000Polynomials: удаление 10 000 полиномов

4.2. Анализ результатов

Тесты

Все функциональные тесты прошли успешно (100% успешных тестов):

1. **Корректность операций**: Все операции (вставка, поиск, удаление, обновление) работают корректно для всех трех структур данных.
2. **Обработка граничных случаев**: Тесты с пустыми таблицами прошли успешно.
3. **Копирование и присваивание**: Конструкторы копирования и операторы присваивания работают корректно, выполняют глубокое копирование.

4. **Балансировка АВЛ-дерева**: Тест BalanceCheck подтвердил, что дерево остается сбалансированным при вставке элементов в произвольном порядке.
5. **Статистика операций**: Счетчики операций увеличиваются при выполнении действий и сбрасываются корректно.

Результаты	Вставка	Поиск	Удаление
Хэш-таблица	420мс	460мс	409мс
Неупорядоченная таблица	2225мс	2275мс	4505мс
АВЛ-дерево	339мс	350мс	391мс

При этом, при использовании алгоритма удаления для упорядоченной таблицы, основанного на удалении элемента из вектора и сдвига остальных элементов время увеличивалось до ~20000мс.

Заключение

В ходе проекта были разработаны шаблонные классы для каждой структуры данных, реализованы основные операции (вставка, поиск, удаление, печать), а также проведено тестирование с использованием фреймворка Google Test.

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/